

Tagueur v3

Philippe
fin septembre 2026

Bref historique

Jusqu'à la version 2, le tagueur LASLA-Collatinus fonctionnait avec un modèle de Markov caché au 3^e ordre (en anglais et par la suite HMM3). Un HMM3 évalue des probabilités sur les séquences de tags¹ et en déduit la meilleure analyse possible pour chaque mot du texte. Cette étape probabiliste est précédée d'une double lemmatisation (en utilisant les données du LASLA et de Collatinus) qui est déterministe et qui donne toutes les analyses connues de la forme trouvée. Si mes souvenirs sont bons, la version 2 a vu une version Daemon destinée à être intégrée à HyperBase.²

La version 3 s'appuie sur Latin-BERT et utilise l'IA pour prédire quelle sera la meilleure analyse. L'IA n'a aucune notion des analyses possibles d'un mot : il faut donc utiliser les analyses trouvées par le tagueur v2 pour contraindre les résultats de l'IA. C'est le travail de réconciliation que je fais avec le programme CSV2APN. Si l'IA prédit une analyse qui n'est pas disponible dans la liste des analyses trouvées, je laisse la solution proposée par le HMM3 et je signale l'incompatibilité à l'utilisateur avec un « warning ». Comme je n'exclus pas que l'IA soit capable de trouver un exemple de substantivation d'un adjectif, j'émet un avertissement que le philologue devrait examiner en priorité.

Initialement, le processus complet était formé de l'appel successif de trois programmes distincts. Tout d'abord, on envoie une requête à un Daemon (issu de la v2) qui lemmatise le texte et prépare les fichiers pour la suite du traitement. Quand ce travail est fini, on lance l'IA qui fait tourner trois modèles différents pour prédire séparément le tag, l'indice d'homonymie et le code de subordination. C'est fait par le code Python `simple_pred.py` ou sa version compilée avec PyInstaller qui est, chez moi, un exécutable Windows `simple_pred.exe`. L'IA prend en entrée un fichier CSV généré par le Daemon et produit trois fichiers CSV avec chacune des prédictions. Le programme CSV2APN reprend ces quatre fichiers CSV et génère les fichiers APN et ZPN³, ainsi que le fichier avec les warnings. Comme je travaillais principalement sur Mac et que la partie Python n'y fonctionne pas, je faisais tourner le Daemon sur mon Mac, j'exportais le CSV produit sur mon PC et j'y effectuais les deux dernières étapes. Un peu compliqué !

J'ai donc opté pour un retour à une interface graphique (GUI en anglais) qui existait pour la v2 et les appels à `simple_pred.exe` et à `CSV2APN.exe` sont intégrés comme des process externes dans le programme et toute la chaîne de traitement est déclenchée par un seul clic. L'intérêt de la version avec GUI est que j'ai pu y remettre la partie d'édition consacrée à la correction de ce qui a été produit par le programme. J'y ai ajouté aussi une possibilité de faire du « pré-processing » pour

-
- 1 Le tag que j'utilise est tiré directement du « code en 9 » du LASLA. Il est composé de trois caractères qui donnent la catégorie, le cas et le nombre ou, pour les formes conjuguées des verbes, la catégorie et le mode (suivi d'un « 1 » s'il s'agit d'un présent). Pour les formes déclinées des verbes (participes et autres), la catégorie « B » est remplacée par « V » qui n'est pas utilisée dans le code en 9.
 - 2 Un Daemon est un programme qui tourne en tâche de fond sur un ordinateur sans se manifester sur l'affichage. C'est une solution intéressante si le programme en question doit charger beaucoup de données au démarrage : l'utilisateur est toujours impatient et ne comprend en général pas pourquoi un programme ne lui donne pas instantanément la réponse. L'utilisateur envoie alors une requête à l'aide d'un programme léger et le Daemon traite cette requête dès qu'il la reçoit. Ici, l'utilité du Daemon est discutable car, pour un texte conséquent, le traitement est souvent plus long que le démarrage.
 - 3 Le format ZPN est une extension de l'APN qui garde toutes les lemmatisations et analyses trouvées pour chaque mot, ainsi que la trace de la ponctuation du texte d'origine. Il est immédiat de tirer l'APN du ZPN (en supprimant beaucoup de lignes), mais on peut aussi utiliser le ZPN pour corriger le résultat proposé par le programme. Paul Pietquin a développé des outils légers pour le faire.

repérer les formes inconnues d'un texte (ce qui suppose une interaction avec le philologue) et, dernièrement, l'intégration de ce que faisait le programme CSV2APN (qui est donc devenu inutile). Des améliorations notables ont aussi été apportées au traitement des formes verbales composées. Il faut donc vérifier qu'elles seront intégrées à la nouvelle version du Daemon. L'appel comme process externe du code initialement en Python peut se discuter, puisque le déclenchement du traitement sera, pour HyperBase, lancé par le serveur web qui peut aussi orchestrer les deux étapes suivantes. Faire le tout en une seule commande est toutefois plus élégant. On peut trouver un intérêt au traitement par étapes dans le cas d'un traitement d'un lot de fichiers textes. En effet, dans ce cas précis, `simple_pred` charge le modèle pour prédire les tags et traite tous les fichiers avant de faire de même avec les modèles pour les indices et pour les codes de subordination. Comme les fichiers décrivant les modèles sont très gros (435 Mo chacun), moins on les charge, mieux c'est.

Les dossiers actuels

J'ai mis en place une organisation de dossiers un peu complexe. L'idée de base est de partager les sources et les données entre les deux versions Daemon et GUI. Comme j'utilise QtCreator, j'ai un fichier projet (*.pro) qui génère au bon moment le fichier Makefile qui convient (à vrai dire, je n'ai jamais regardé en détails ce qui est écrit dans ce fichier Makefile). Or ce nom « Makefile » étant figé, les divers projets doivent être placés dans des dossiers différents : d'où un dossier `Tagueur_Daemon` et un autre `Tagueur_GUI`, côte à côte, avec également le dossier `src` et d'autres dossiers pour les utilitaires (`Client_C11` et `CSV2APN`, le premier étant utilisé pour communiquer avec le Daemon et le second est devenu théoriquement inutile mais encore nécessaire parce qu'appelé). Tous ces dossiers ainsi que le dossier cible, `Tagueur_IA`, sont placés dans le grand dossier `Tagueur_v3`. Le dossier `Tagueur_IA` va recevoir les exécutables (et tout ce qui est nécessaire à l'exécution du programme). Ce dossier contient aussi les données (dossier `data`), les modèles (dossier `Models`) et le programme Python compilé `simplePred.exe`. En réalité, je viens de constater que je n'ai jamais compilé le Daemon sur mon PC. Du coup, je ne sais pas s'il est même compilable ou si j'ai laissé tout en plan (en bref, je dois revoir ce qui a subsisté et ce qui a changé).

Commentaires sur les sources

Je ne suis pas expert de C/C++, ni de Qt, ni des fichiers projets qui sont utilisés par QtCreator. Toutefois, ces fichiers projets simplifient nettement la gestion d'un ensemble de sources. Ils contiennent en particulier le nom du fichier exécutable à créer et les noms des dossiers où ranger les divers fichiers produits. Par exemple, pour le `Tagueur_GUI` :

```
TARGET = LASLA_tagger_3
TEMPLATE = app
OBJECTS_DIR= obj/
MOC_DIR = moc/
DESTDIR = ../Tagueur_IA
```

Pour le déploiement, on a aussi des commandes spéciales qui servent à mettre dans un même dossier tous les ingrédients (essentiellement les dll) nécessaires au fonctionnement autonome de l'exécutable (même sur une machine qui n'a jamais vu Qt). Ce qui n'est pas bien clair (pour moi) est que le contenu du fichier projet ne suffit pas à déterminer le comportement. En effet, il faut déclarer une étape de déploiement dans le volet « project » de QtCreator pour qu'elle devienne effective :

```
deploy.commands = windeployqt ../Tagueur_IA/LASLA_tagger_3.exe
QMAKE_EXTRA_TARGETS += deploy
```

Dans le fichier projet, il y a la liste des fichiers sources (et headers) qui sont nécessaires. Par exemple, toujours pour le `Tagueur_GUI`, on a :

```
SOURCES += ../src/main.cpp\
           ../src/mainwindow.cpp\
           ../src/serveur.cpp\
```

```
../src/fiche.cpp\  
../src/mot.cpp\  
../src/traitements.cpp\  
../src/ch.cpp \  
../src/irregs.cpp \  
../src/lemme.cpp \  
../src/modele.cpp \  
../src/lasla.cpp \  
../src/lemCore.cpp
```

Pour l'instant, je garde le module "serveur"

même si je ne vais pas l'utiliser. 28/08/2024

Les modules ch, irregs, lemme, modele et lemCore sont issus de Collatinus (c'est le moteur de la lemmatisation). Les modules fiche, mot et lasla sont maintenant le cœur du traitement complet des textes. Pour des raisons historiques, le module MainWindow, qui gère la GUI et orchestre les opérations, a été coupé en deux fichiers mainwindow et traitement. Il faudrait probablement que je fasse du ménage là-dedans, d'autant qu'une grande partie du traitement se fait maintenant dans le module lasla (qui était ridiculement réduit dans Collatinus). Comme je l'ai écrit il y a deux ans, je garde le module serveur sans l'utiliser. Le code main.cpp sert à lancer le gestionnaire de fenêtre mainwindow.

Dans le Daemon, main.cpp est remplacé par main_daemon.cpp qui appelle le serveur. Les fichiers associés à la GUI (mainwindow et traitement) disparaissent.

Dans la même veine, on a client_main.cpp qui sert à communiquer avec le Daemon et main_CSV2APN.cpp qui utilise le module token pour réconcilier les prédictions de l'IA avec les possibilités identifiées.